

Assignment 3

(LAB 1: Display Density Metrics & Image Array Mechanics in NumPy)

Student Name: Partab Rai

Roll Number: 2k24/CSE/117

Program: Computer Science (Part III P.E)

Submission Date: September 11, 2026

Lab Deliverables Checklist

✓ **Task 1: Display Pixel Density (PPI/DPI) Calculator**

- Interactive CLI script for PPI calculation & aspect ratio reduction.
- Automated density classification rules (Low, Medium, High).
- Console outputs & execution screenshots attached.

✓ **Task 2: Environment Setup & Synthetic Image Matrix Creation**

- $300 \times 400 \times 3$ uint8 3D NumPy matrix constructed from scratch.
- Spatial quadrant slicing for pure Red, Green, Blue, and White colors.
- Array properties (shape, dtype, size, nbytes) logged.

✓ **Task 3: Channel Slicing & Isolation**

- 2D channel intensity grid extraction using Axis 2 slicing.
- Single-channel color isolations rendered alongside grayscale views.
- 2×3 Matplotlib subplot visualization rendered and attached.

✓ **Task 4: Spatial Downsampling & Pixelation via Striding**

- Matrix downsampling via step-slicing (`img[:, :N, :N, :]`) for $N = 8$.
- Re-expansion using `np.repeat()` to visualize blocky pixelation.
- Spatial dimension drop and 98.44% memory savings calculated.

✓ **Short Answer Theory Question**

- Mathematical explanation for 33.33% RAM increase in RGBA vs RGB.

Task 1: Display Pixel Density (PPI/DPI) Calculator

Objective: Write a Python script that calculates screen parameters and classifies display density based on physical screen attributes.

Requirements:

1. Prompt the user to input horizontal pixels (W_{px}), vertical pixels (H_{px}), and physical screen diagonal (D_{inches}).
2. Calculate and display:
 - Total pixel count ($W_{\text{px}} \times H_{\text{px}}$).
 - Simplified aspect ratio ($W : H$) using the greatest common divisor (`math.gcd`).
 - Screen DPI/PPI rounded to two decimal places.
3. Classify the display density into one of the following categories:
 - $\text{DPI} < 100$: **Low Density** (Standard Monitor)
 - $100 \leq \text{DPI} \leq 200$: **Medium Density** (HD Display)
 - $\text{DPI} > 200$: **High Density** (Retina / Mobile)

Sample Execution:

Sample Input

Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

Sample Output

```
--- DISPLAY METRICS ANALYSIS ---  
Total Pixel Count : 2,073,600 pixels  
Aspect Ratio : 16:9  
Calculated DPI : 91.79 DPI  
Density Category : Low Density (Standard Monitor)
```

Source Code (Task 1)

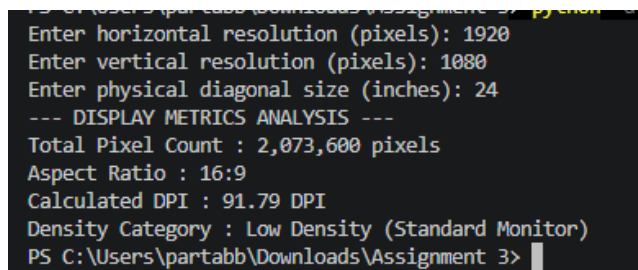
```
1 import math  
2  
3 Wpx = int(input("Enter horizontal resolution (pixels): "))  
4 Hpx = int(input("Enter vertical resolution (pixels): "))  
5 Dinches = float(input("Enter physical diagonal size (inches): "))  
6  
7 total_pixels = Wpx * Hpx  
8 gcd = math.gcd(Wpx, Hpx)  
9 aspect_width = Wpx // gcd  
10 aspect_height = Hpx // gcd  
11 dpi = ((Wpx**2 + Hpx**2) ** 0.5) / Dinches
```

```

12
13 if dpi < 100:
14     density_category = "Low Density (Standard Monitor)"
15 elif dpi <= 200:
16     density_category = "Medium Density (HD Display)"
17 else:
18     density_category = "High Density (Retina / Mobile)"
19
20 print("--- DISPLAY METRICS ANALYSIS ---")
21 print(f"Total Pixel Count : {total_pixels:,} pixels")
22 print(f"Aspect Ratio       : {aspect_width}:{aspect_height}")
23 print(f"Calculated DPI      : {dpi:.2f} DPI")
24 print(f"Density Category   : {density_category}")

```

Actual Task 1 Output Screenshots

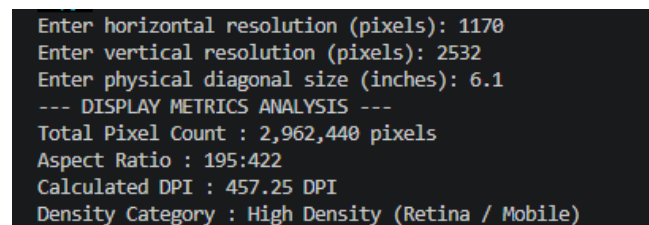


```

PS C:\Users\partabb\Downloads\Assignment 3\python 3>
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24
--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
PS C:\Users\partabb\Downloads\Assignment 3>

```

Figure 1: Desktop Test Screenshot



```

Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1
--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio : 195:422
Calculated DPI : 457.25 DPI
Density Category : High Density (Retina / Mobile)

```

Figure 2: Mobile Test Screenshot

Task 2: Environment Setup & Synthetic Image Matrix Creation

Objective: Install Python graphics packages and construct a 3D image matrix from scratch.

Requirements:

1. Install required packages: `pip install numpy pillow matplotlib`.
2. Create a synthetic $300 \times 400 \times 3$ image array initialized with zeros using `np.zeros()` with data type `np.uint8`.
3. Fill the four quadrants using spatial array slicing `[row_start:row_end, col_start:col_end]`:
 - Top-Left: Pure Red `[255, 0, 0]`, Top-Right: Pure Green `[0, 255, 0]`
 - Bottom-Left: Pure Blue `[0, 0, 255]`, Bottom-Right: White `[255, 255, 255]`
4. Print matrix properties: `.shape`, `.dtype`, `.size`, and `.nbytes`.

Sample Execution:

Sample Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360,000 values
Memory Footprint : 360,000 bytes (351.56 KB)
```

Source Code (Task 2)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 image = np.zeros((300, 400, 3), dtype=np.uint8)
5 image[:150, :200] = [255, 0, 0]      # Top-Left: Red
6 image[:150, 200:] = [0, 255, 0]      # Top-Right: Green
7 image[150:, :200] = [0, 0, 255]      # Bottom-Left: Blue
8 image[150:, 200:] = [255, 255, 255]  # Bottom-Right: White
9
10 print("--- SYNTHETIC MATRIX METRICS ---")
11 print(f"Array Shape (H, W, C) : {image.shape}")
12 print(f>Data Type           : {image.dtype}")
13 print(f>Total Elements      : {image.size:,} values")
14 print(f>Memory Footprint    : {image.nbytes:,} bytes ({image.nbytes /
15     1024:.2f} KB)")
16
16 plt.imshow(image)
17 plt.axis('off')
18 plt.savefig('outputs/task2_synthetic.png', bbox_inches='tight', pad_inches=0)
19 plt.show()
```

Actual Task 2 Output Image

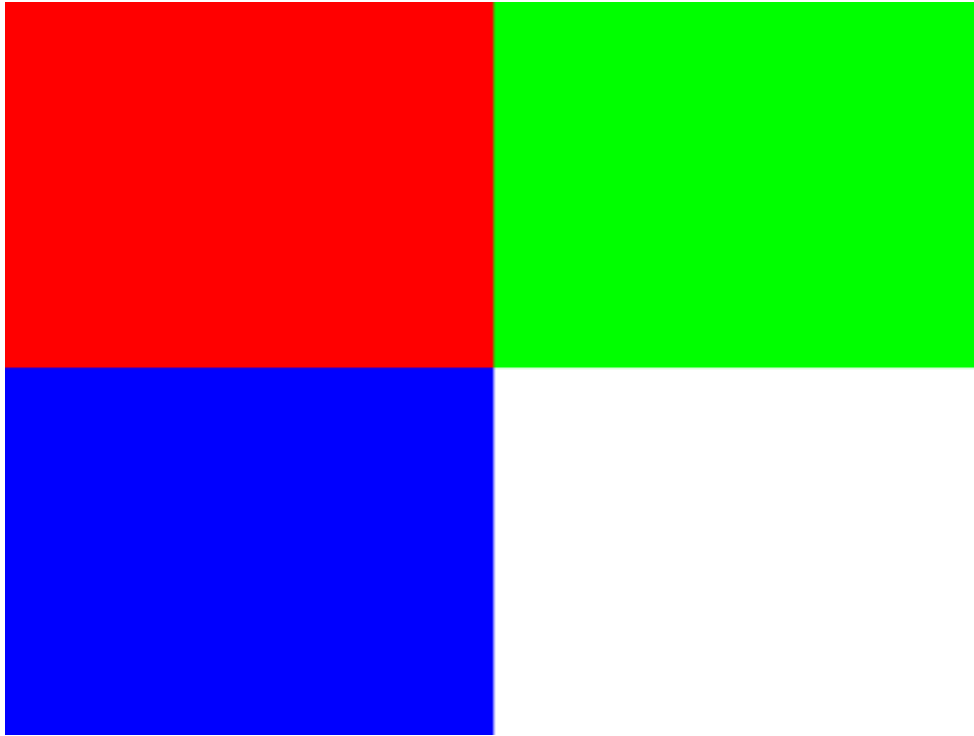


Figure 3: Task 2: Generated Synthetic Quadrant Matrix (`task2_synthetic.png`)

Task 3: Channel Slicing & Isolation

Objective: Load an image, extract individual 2D channel intensity grids, and isolate single-channel color views in a 2×3 layout.

Requirements:

1. Load an input image (`sample.jpg`) into a NumPy array.
2. Extract 2D intensity grids for Red, Green, and Blue channels using Axis 2 slicing.
3. Construct three separate 3D color arrays where only one channel is active.
4. Render in a 2×3 subplot layout (Top Row: Color Isolations, Bottom Row: Grayscale Intensities).

Sample Execution:

Sample Output

```
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (1080, 1920, 3)
Red Channel 2D Shape : (1080, 1920) | Mean Intensity: 176.51
Green Channel 2D Shape : (1080, 1920) | Mean Intensity: 220.18
Blue Channel 2D Shape : (1080, 1920) | Mean Intensity: 235.71
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
```

Source Code (Task 3)

```
1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 image = np.array(Image.open("sample.jpg").convert("RGB"))
6
7 red_channel = image[:, :, 0]
8 green_channel = image[:, :, 1]
9 blue_channel = image[:, :, 2]
10
11 red_only = np.zeros_like(image); red_only[:, :, 0] = red_channel
12 green_only = np.zeros_like(image); green_only[:, :, 1] = green_channel
13 blue_only = np.zeros_like(image); blue_only[:, :, 2] = blue_channel
14
15 fig, axes = plt.subplots(2, 3, figsize=(12, 8))
16 axes[0, 0].imshow(red_only); axes[0, 0].set_title("Red-Only")
17 axes[0, 1].imshow(green_only); axes[0, 1].set_title("Green-Only")
18 axes[0, 2].imshow(blue_only); axes[0, 2].set_title("Blue-Only")
19
20 axes[1, 0].imshow(red_channel, cmap="gray"); axes[1, 0].set_title("Red
    Intensity")
21 axes[1, 1].imshow(green_channel, cmap="gray"); axes[1, 1].set_title("Green
```

```

    Intensity")
22 axes[1, 2].imshow(blue_channel, cmap="gray"); axes[1, 2].set_title("Blue
    Intensity")
23
24 for ax in axes.flat:
25     ax.axis("off")
26
27 plt.tight_layout()
28 plt.savefig("outputs/task3_subplots.png", bbox_inches="tight")
29 plt.show()

```

Actual Task 3 Output Image

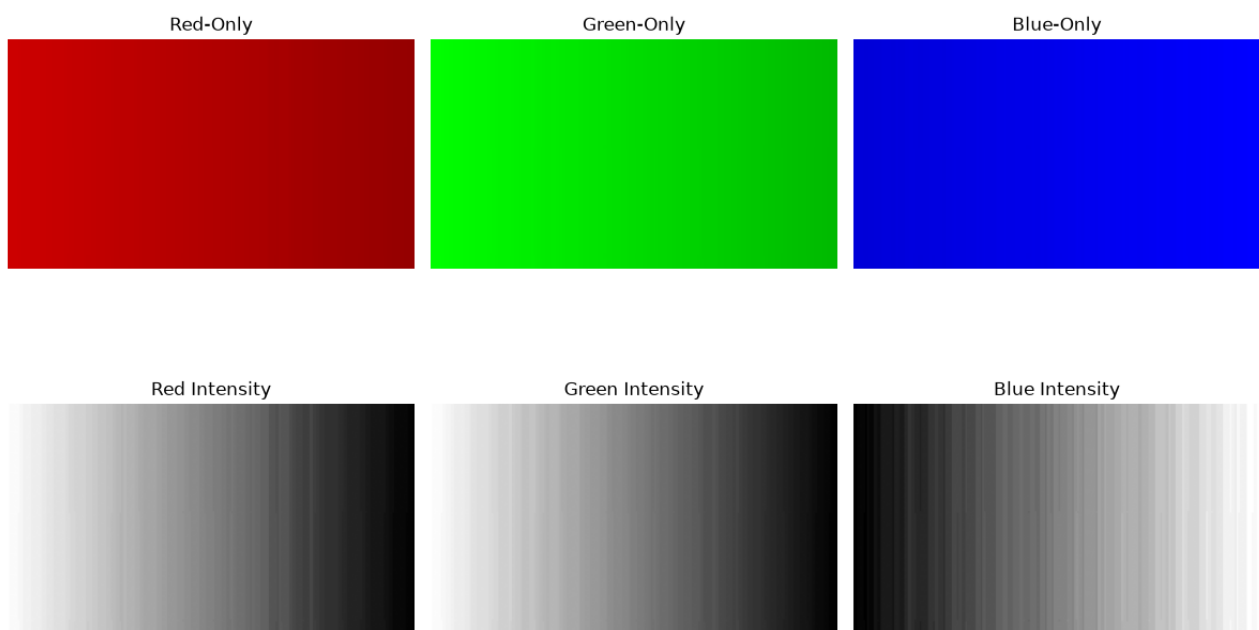


Figure 4: Task 3: Matplotlib 2x3 Subplot Channel Grid (task3_subplots.png)

Task 4: Spatial Downsampling & Pixelation via Striding

Objective: Perform spatial downsampling directly in NumPy using slice steps and analyze memory reduction.

Requirements:

1. Downsample an input image matrix by taking every N -th pixel using NumPy striding (`img[::N, ::N, :]`) for $N = 8$.
2. Re-expand downsampled array back to original dimensions using `np.repeat()` to observe blocky pixelation.
3. Compute spatial dimension reduction percentage and memory footprint savings.

Sample Execution:

Sample Output

```
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape : (1080, 1920, 3) | Memory: 6,220,800 bytes
Downsampled Shape : (135, 240, 3) | Memory: 97,200 bytes
Re-expanded Shape : (1080, 1920, 3) | Visual: Blocky Pixelation
Dimension Reduction: 87.50% reduction per axis
Memory Savings : 98.44% data reduction
```

Source Code (Task 4)

```
1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 image = np.array(Image.open("sample.jpg").convert("RGB"))
6 N = 8
7
8 downsampled = image[::N, ::N, :]
9 reexpanded = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)
10 reexpanded = reexpanded[:image.shape[0], :image.shape[1], :]
11
12 fig, axes = plt.subplots(1, 2, figsize=(12, 6))
13 axes[0].imshow(image)
14 axes[0].set_title(f"Original Image\nShape: {image.shape}")
15 axes[0].axis('off')
16
17 axes[1].imshow(reexpanded)
18 axes[1].set_title(f"Re-expanded Pixelated Image (N={N})\nShape: {reexpanded.\n    shape}")
19 axes[1].axis('off')
20
21 plt.tight_layout()
```

```
22 plt.savefig('outputs/task4_downsampled.png', bbox_inches='tight')
23 plt.show()
```

Actual Task 4 Output Image

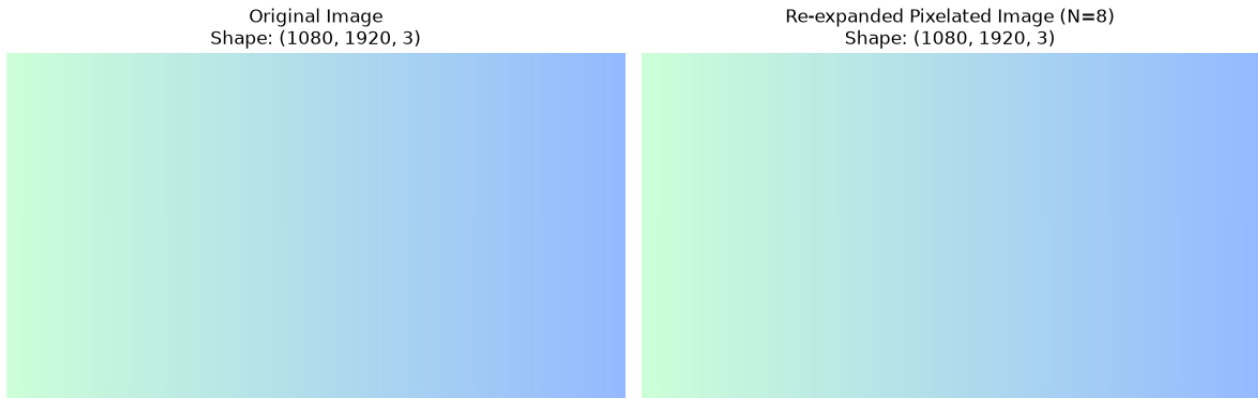


Figure 5: Task 4: Spatial Downsampling & Pixelation Comparison (`task4_downsampled.png`)

Short Answer Theory Question

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Standard RGB images store 3 color channels ($C = 3$) per pixel corresponding to Red, Green, and Blue. Adding an Alpha channel for transparency increases the channel dimension to 4 ($C = 4$).

Because memory footprint scales linearly with total array elements ($H \times W \times C$), adding 1 additional channel to 3 channels yields a memory increase factor calculated as:

$$\frac{4 - 3}{3} = \frac{1}{3} \approx 33.33\%$$

Therefore, an RGBA image array uses exactly 33.33% more memory in RAM than an RGB array of identical height and width dimensions.